

Vertex AI：使用 Sklearn 的自訂預測處理常式，預先處理及後續處理預測資料

來源：<https://codelabs.developers.google.com/vertex-cpr-sklearn?hl=zh-tw>

步驟數：8

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [用途總覽](#)
4. [設定環境](#)
5. [編寫訓練程式碼](#)
6. [使用 CPR 模型伺服器建構自訂服務容器](#)
7. [將模型部署至 Vertex AI](#)
8. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將瞭解如何在 Vertex AI 上使用自訂預測處理常式，編寫自訂的預先處理和後續處理邏輯。雖然本範例使用 Scikit-learn，但自訂預測處理常式可搭配其他 Python 機器學習架構，例如 XGBoost、PyTorch 和 TensorFlow。

課程內容

內容如下：

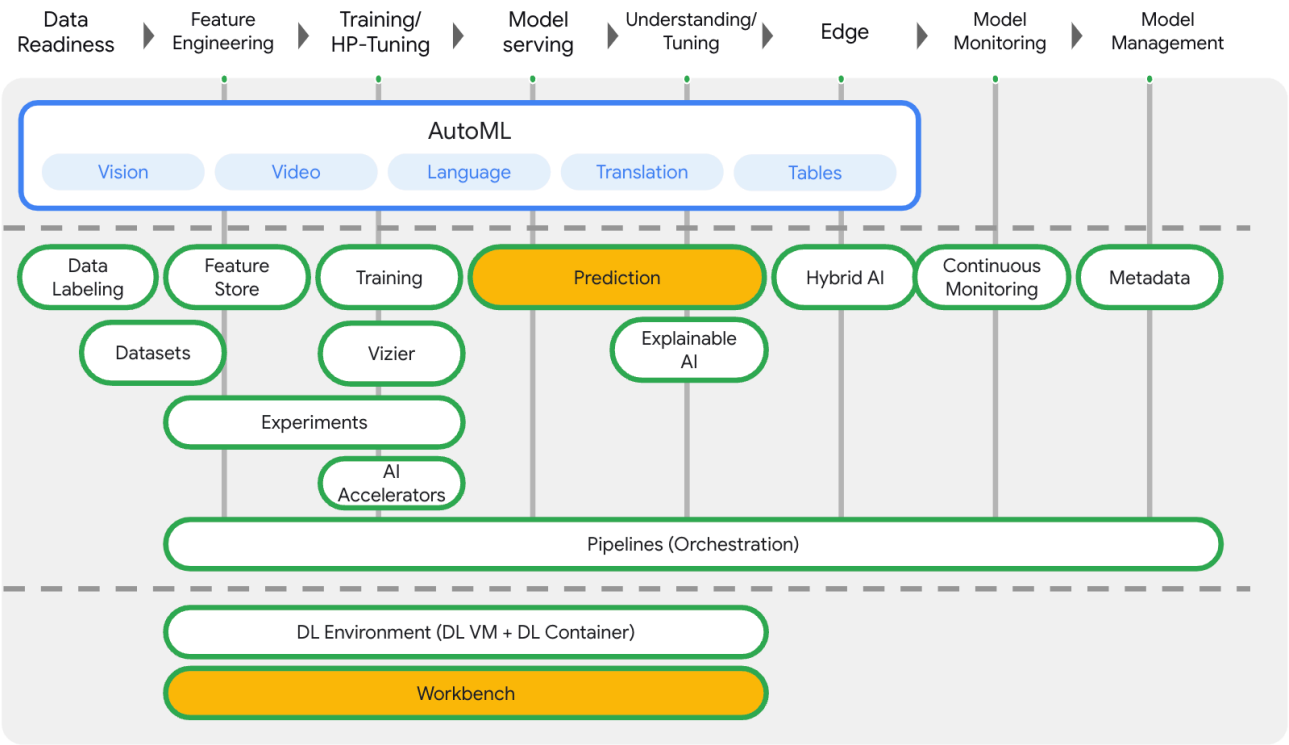
- 使用自訂預測處理常式撰寫自訂預測邏輯
- 在本機測試自訂服務容器和模型
- 在 Vertex AI Predictions 上測試自訂服務容器

在 Google Cloud 上執行這個實驗室的總費用約為 **\$1** 美元。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。Vertex AI 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於**Predictions**和**Workbench**。



3. 用途總覽

用途

在本實驗室中，您將建構隨機森林迴歸模型，根據切工、淨度和大小等屬性預測鑽石價格。

您將編寫自訂預先處理邏輯，檢查服務時的資料是否為模型預期的格式。您也會編寫自訂後續處理邏輯，將預測結果四捨五入並轉換為字串。如要編寫這項邏輯，請使用自訂預測處理常式。

自訂預測處理常式簡介

[Vertex AI 預先建構的容器](#)會執行機器學習架構的預測作業，處理預測要求。在自訂預測處理常式推出前，如果您想在執行預測前預先處理輸入內容，或在傳回結果前對模型預測進行後處理，就必須建構自訂容器。

如要建構自訂服務容器，必須編寫 HTTP 伺服器，將訓練好的模型包裝起來、將 HTTP 要求轉換為模型輸入內容，並將模型輸出內容轉換為回應。

透過自訂預測處理常式，Vertex AI 會為您提供服務相關元件，讓您專注於模型和資料轉換。

步驟 4 · 約 10 分鐘

4. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。您需要這項資訊才能建立筆記本執行個體。

步驟 2：啟用 Artifact Registry API

前往 [Artifact Registry](#)，然後選取「啟用」（如果尚未啟用）。您將使用這個工具建立自訂服務容器。

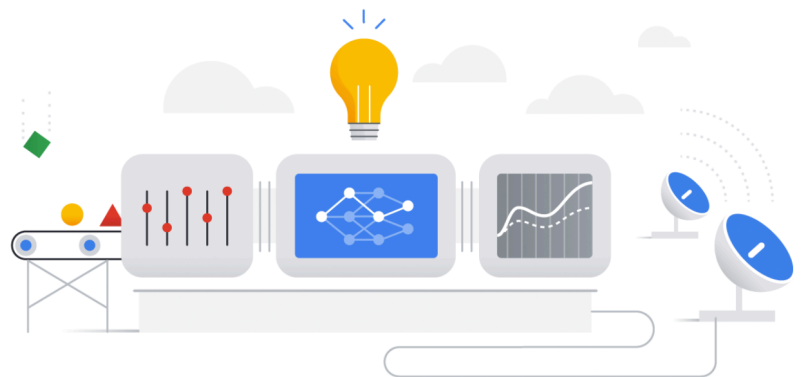
步驟 3：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

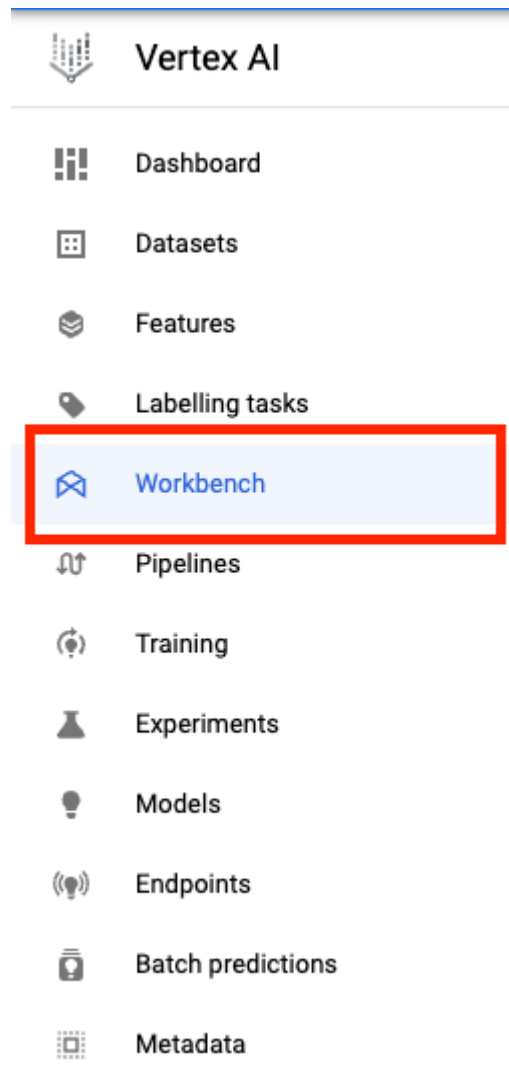
Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 4：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



如果尚未啟用 Notebooks API，請先啟用。



Notebooks API

Google Enterprise API

Notebooks API is used to manage notebook resources in Google Cloud.

ENABLE

TRY THIS API [↗](#)

OVERVIEW

PRICING

DOCUMENTATION

Overview

Notebooks API is used to manage notebook resources in Google Cloud.

Additional details

Type: [SaaS & APIs](#)

Last updated: 15/09/2021

Category: [Google Enterprise APIs](#)

Service name: notebooks.googleapis.com

啟用後，按一下「執行個體」，然後選取「建立新項目」。

接受預設選項，然後點選「建立」。

執行個體就緒後，按一下「OPEN JUPYTERLAB」開啟執行個體。

步驟 5 · 約 10 分鐘

5. 編寫訓練程式碼

步驟 1：建立 Cloud Storage bucket

您會將模型和前處理構件儲存在 Cloud Storage bucket 中。如果您已在專案中建立要使用的 bucket，可以略過這個步驟。
從啟動器開啟新的終端機工作階段。



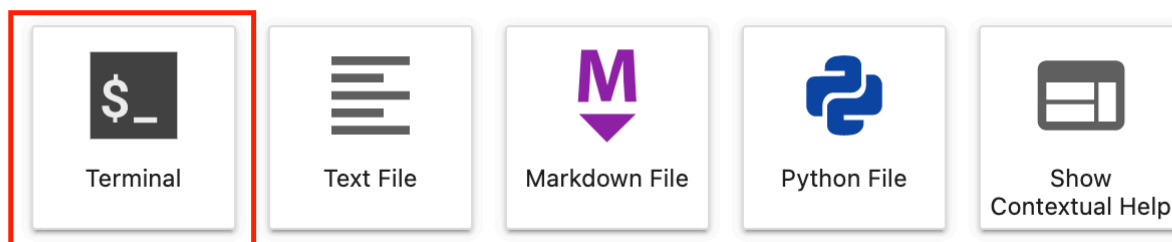
Notebook



Console



Other



在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

接著，在終端機中執行下列指令，在專案中建立新的 bucket。

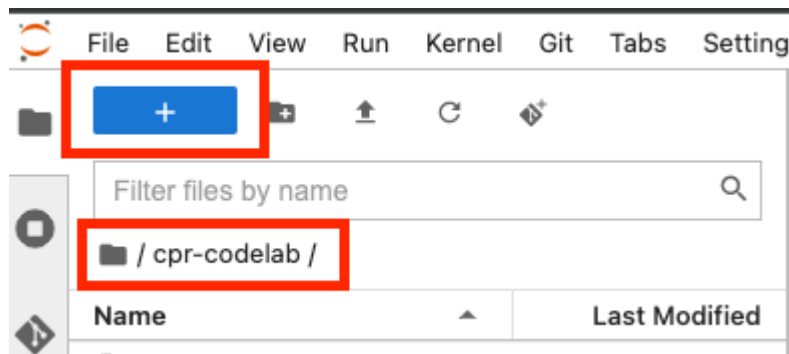
```
BUCKET="gs://${PROJECT_ID}-cpr-bucket"  
gsutil mb -l us-central1 $BUCKET
```

步驟 2：訓練模型

在終端機中，建立名為 `cpr-codelab` 的新目錄，然後 `cd` 到該目錄。

```
mkdir cpr-codelab  
cd cpr-codelab
```

在檔案瀏覽器中，前往新的 `cpr-codelab` 目錄，然後使用啟動器建立名為 `task.ipynb` 的新 Python 3 筆記本。



您的 `cpr-codelab` 目錄現在應如下所示：

```
+ cpr-codelab/  
  + task.ipynb
```

在筆記本中貼上下列程式碼。

首先，請編寫 `requirements.txt` 檔案。

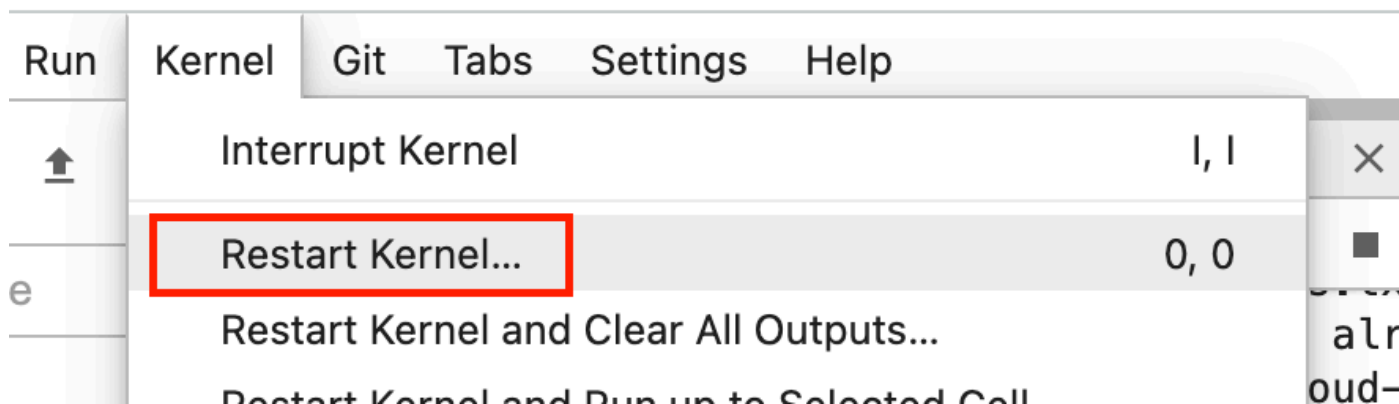
```
%%writefile requirements.txt  
fastapi  
uvicorn==0.17.6  
joblib~=1.0  
numpy~=1.20  
scikit-learn>=1.2.2  
pandas  
google-cloud-storage>=1.26.0,<2.0.0dev  
google-cloud-aiplatform[prediction]>=1.16.0
```

您部署的模型預先安裝的依附元件，與筆記本環境不同。因此，您需要在 `requirements.txt` 中列出模型的所有依附元件，然後使用 `pip` 在筆記本中安裝完全相同的依附元件。稍後，您將在本機測試模型，然後部署至 Vertex AI，再次確認環境是否相符。

在筆記本中透過 `Pip` 安裝依附元件。

```
!pip install -U --user -r requirements.txt
```

請注意，pip 安裝完成後，您必須重新啟動核心。



接著，請建立目錄，用於儲存模型和前處理構件。

```
USER_SRC_DIR = "src_dir"
!mkdir $USER_SRC_DIR
!mkdir model_artifacts

# copy the requirements to the source dir
!cp requirements.txt $USER_SRC_DIR/requirements.txt
```

您的 `cpr-codelab` 目錄現在應如下所示：

```
+ cpr-codelab/
  + model_artifacts/
  + src_dir/
    + requirements.txt
  + task.ipynb
  + requirements.txt
```

目錄結構設定完成後，就可以開始訓練模型了！

首先，匯入程式庫。

```
import seaborn as sns
import numpy as np
import pandas as pd

from sklearn import preprocessing
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer

import joblib
import logging

# set logging to see the docker container logs
logging.basicConfig(level=logging.INFO)
```

然後定義下列變數。請務必將 `PROJECT_ID` 替換為專案 ID，並將 `BUCKET_NAME` 替換為您在上一個步驟中建立的 bucket。

```
REGION = "us-central1"
MODEL_ARTIFACT_DIR = "sklearn-model-artifacts"
REPOSITORY = "diamonds"
IMAGE = "sklearn-image"
MODEL_DISPLAY_NAME = "diamonds-cpr"

# Replace with your project
PROJECT_ID = "{PROJECT_ID}"

# Replace with your bucket
BUCKET_NAME = "gs://{BUCKET_NAME}"
```

從 Seaborn 程式庫載入資料，然後建立兩個 DataFrame，一個包含特徵，另一個包含標籤。

```
data = sns.load_dataset('diamonds', cache=True, data_home=None)

label = 'price'

y_train = data['price']
x_train = data.drop(columns=['price'])
```

我們來看看訓練資料。您會看到每一列都代表一個鑽石。

```
x_train.head()
```

以及對應的價格標籤。

```
y_train.head()
```

現在，請定義 sklearn [資料欄轉換](#)，對類別特徵進行 one-hot 編碼，並縮放數值特徵

```
column_transform = make_column_transformer(
    (preprocessing.OneHotEncoder(), [1,2,3]),
    (preprocessing.StandardScaler(), [0,4,5,6,7,8]))
```

定義隨機樹系模型

```
regr = RandomForestRegressor(max_depth=10, random_state=0)
```

接著，建立 [sklearn 管道](#)。也就是說，傳送至這個管道的資料會先經過編碼/縮放，再傳送至模型。

```
my_pipeline = make_pipeline(column_transform, regr)
```

使用訓練資料擬合管道

```
my_pipeline.fit(x_train, y_train)
```

請試用模型，確保模型運作正常。對模型呼叫 `predict` 方法，並傳入測試樣本。

```
my_pipeline.predict([[0.23, 'Ideal', 'E', 'SI2', 61.5, 55.0, 3.95, 3.98, 2.43]])
```

現在我們可以將管道儲存至 `model_artifacts` 目錄，並複製到 Cloud Storage bucket。

```
joblib.dump(my_pipeline, 'model_artifacts/model.joblib')

!gsutil cp model_artifacts/model.joblib {BUCKET_NAME}/{MODEL_ARTIFACT_DIR}/
```

步驟 3：儲存預先處理構件

接下來，您將建立前處理構件。模型伺服器啟動時，這個構件會載入自訂容器。前處理構件幾乎可以是任何形式 (例如 Pickle 檔案)，但在此情況下，您會將字典寫入 JSON 檔案。

```
clarity_dict={"Flawless": "FL",
              "Internally Flawless": "IF",
              "Very Very Slightly Included": "VVS1",
              "Very Slightly Included": "VS2",
              "Slightly Included": "SI2",
              "Included": "I3"}
```

訓練資料中的 `clarity` 特徵一律為縮寫形式 (即「FL」而非「Flawless」)。在放送時，我們希望檢查這項功能的資料是否也經過縮寫。這是因為模型知道如何對「FL」進行單熱編碼，但不知道如何對「Flawless」進行單熱編碼。您稍後會編寫這項自訂前處理邏輯。但目前只要將這個對照表儲存至 JSON 檔案，然後寫入 Cloud Storage bucket 即可。

```
import json
with open("model_artifacts/preprocessor.json", "w") as f:
    json.dump(clarity_dict, f)

!gsutil cp model_artifacts/preprocessor.json {BUCKET_NAME}/{MODEL_ARTIFACT_DIR}/
```

您的本機 `cpr-codelab` 目錄現在應如下所示：

```
+ cpr-codelab/
  + model_artifacts/
    + model.joblib
    + preprocessor.json
  + scr_dir/
    + requirements.txt
  + task.ipynb
  + requirements.txt
```

6. 使用 CPR 模型伺服器建構自訂服務容器

模型訓練完成並儲存預先處理構件後，現在可以建構自訂服務容器。通常建立供應容器時，需要撰寫模型伺服器程式碼。不過，使用自訂預測處理常式時，Vertex AI Predictions 會為您產生模型伺服器，並建構自訂容器映像檔。

自訂放送容器包含下列 3 段程式碼：

1. 模型伺服器 (SDK 會自動產生這個伺服器，並儲存在 `src_dir/` 中)

- 代管模型的 HTTP 伺服器
- 負責設定路徑/連接埠等。

2. 要求處理常式

- 負責處理要求的網路伺服器相關作業，例如將要求主體反序列化、將回應序列化、設定回應標頭等。
- 在本例中，您將使用 SDK 中提供的預設處理常式

```
google.cloud.aiplatform.prediction.handler.PredictionHandler
```

3. 預測因子

- 負責處理預測要求的機器學習邏輯。

您可以根據用途需求自訂這些元件。在本範例中，您只會實作預測器。

預測者負責處理預測要求的機器學習邏輯，例如自訂預先處理和後續處理。如要編寫自訂預測邏輯，請將 [Vertex AI Predictor](#) 介面設為子類別。

這個版本的自訂預測常式隨附可重複使用的 XGBoost 和 Sklearn 預測器，但如果您需要使用其他框架，可以透過子類別化基礎預測器來建立自己的預測器。

請參閱下方的 [Sklearn 預測器](#) 範例。如要建構這個自訂模型伺服器，您只需要編寫這些程式碼。

```
class SklearnPredictor(Predictor):
    """Default Predictor implementation for Sklearn models."""

    def __init__(self):
        return

    def load(self, artifacts_uri: str):
        prediction_utils.download_model_artifacts(artifacts_uri)
        self._model = joblib.load("model.joblib")

    def preprocess(self, prediction_input: dict) -> np.ndarray:
        instances = prediction_input["instances"]
        return np.asarray(instances)

    def predict(self, instances: np.ndarray) -> np.ndarray:
        return self._model.predict(instances)

    def postprocess(self, prediction_results: np.ndarray) -> dict:
        return {"predictions": prediction_results.tolist()}
```

在筆記本中貼上下列程式碼，將 `SklearnPredictor` 子類別化，並寫入 `src_dir/` 中的 Python 檔案。請注意，在本範例中，我們只會自訂 `load`、`preprocess` 和 `postprocess` 方法，不會自訂 `predict` 方法。

```
%%writefile $USER_SRC_DIR/predictor.py

import joblib
import numpy as np
import json

from google.cloud import storage
from google.cloud.aiplatform.prediction.sklearn.predictor import SklearnPredictor

class CprPredictor(SklearnPredictor):

    def __init__(self):
        return

    def load(self, artifacts_uri: str) -> None:
        """Loads the sklearn pipeline and preprocessing artifact."""

        super().load(artifacts_uri)

        # open preprocessing artifact
        with open("preprocessor.json", "rb") as f:
            self._preprocessor = json.load(f)

    def preprocess(self, prediction_input: np.ndarray) -> np.ndarray:
        """Performs preprocessing by checking if clarity feature is in abbreviated form."""

        inputs = super().preprocess(prediction_input)

        for sample in inputs:
            if sample[3] not in self._preprocessor.values():
                sample[3] = self._preprocessor[sample[3]]
        return inputs

    def postprocess(self, prediction_results: np.ndarray) -> dict:
        """Performs postprocessing by rounding predictions and converting to str."""

        return {"predictions": [f"${value}" for value in np.round(prediction_results)]}
```

接著我們將深入探討這些方法。

- `load` 方法會載入前處理構件，在本例中，該構件是將鑽石淨度值對應至縮寫的字典。
- `preprocess` 方法會使用該構件，確保在放送時，清晰度功能會採用縮寫格式。如果不是，系統會將完整字串轉換為縮寫。
- `postprocess` 方法會傳回預測值，並以 \$ 符號和四捨五入的值表示。

請注意，這個範例的主要目標是說明如何自訂這些不同方法，而非建構實際模型或服務管道。

接著，請使用 Vertex AI Python SDK 建構圖片。使用自訂預測處理常式時，系統會為您產生 Dockerfile 並建構映像檔。

```
from google.cloud import aiplatform

aiplatform.init(project=PROJECT_ID, location=REGION)

import os

from google.cloud.aiplatform.prediction import LocalModel

from src_dir.predictor import CprPredictor # Should be path of variable $USER_SRC_DIR

local_model = LocalModel.build_cpr_model(
    USER_SRC_DIR,
    f"{REGION}-docker.pkg.dev/{PROJECT_ID}/{REPOSITORY}/{IMAGE}",
    predictor=CprPredictor,
    requirements_path=os.path.join(USER_SRC_DIR, "requirements.txt"),
)
```

編寫測試檔，其中包含兩個用於預測的樣本。其中一個執行個體有縮寫的清晰度名稱，但另一個需要先轉換。

```
import json

sample = {"instances": [
    [0.23, 'Ideal', 'E', 'VS2', 61.5, 55.0, 3.95, 3.98, 2.43],
    [0.29, 'Premium', 'J', 'Internally Flawless', 52.5, 49.0, 4.00, 2.13, 3.11]]}

with open('instances.json', 'w') as fp:
    json.dump(sample, fp)
```

部署本機模型，在本機測試容器。

在本範例中，我們使用本機模型和前處理構件，但您也可以指定 Cloud Storage 路徑做為 `artifact_uri`。在這種情況下，您需要[按照這個範例所示](#)設定憑證。

```
with local_model.deploy_to_local_endpoint(
    artifact_uri = 'model_artifacts/', # local path to artifacts
) as local_endpoint:
    predict_response = local_endpoint.predict(
        request_file='instances.json',
        headers={"Content-Type": "application/json"},
    )

    health_check_response = local_endpoint.run_health_check()
```

您可以透過下列方式查看預測結果：

predict_response.content

步驟 7 · 約 0 分鐘

7. 將模型部署至 Vertex AI

您已在本機測試容器，現在要將映像檔推送至 Artifact Registry，並將模型上傳至 Vertex AI Model Registry。

首先，請設定 Docker 來存取 Artifact Registry。

```
!gcloud artifacts repositories create {REPOSITORY} --repository-format=docker \
--location=us-central1 --description="Docker repository"

!gcloud auth configure-docker {REGION}-docker.pkg.dev --quiet
```

然後推送映像檔。

```
local_model.push_image()
```

然後上傳模型。

```
model = aiplatform.Model.upload(local_model = local_model,
                                display_name=MODEL_DISPLAY_NAME,
                                artifact_uri=f"{BUCKET_NAME}/{MODEL_ARTIFACT_DIR}",)
```

模型上傳完畢後，您應該會在控制台中看到模型：

Models

 CREATE  IMPORT

Models are built from your data sets or unmanaged data sources. There are many different types of machine learning models available on Vertex AI, depending on your use case and level of experience with machine learning. [Learn more](#)

Region

us-central1 (Iowa) ▼ ?

Filter

Enter a property name

Name	Deployment status	Description	Default version
diamonds-cpr	—	—	1

接著部署模型，即可用於線上預測。自訂預測處理常式也適用於批次預測，因此如果您的用途不需要線上預測，就不必部署模型。

```
endpoint = model.deploy(machine_type="n1-standard-2")
```

最後，取得預測結果，測試已部署的模型。

```
endpoint.predict(instances=[[0.23, 'Ideal', 'E', 'VS2', 61.5, 55.0, 3.95, 3.98, 2.43]])
```

🎉 恭喜！🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

- 使用自訂預測處理常式，撰寫自訂預先處理和後續處理邏輯

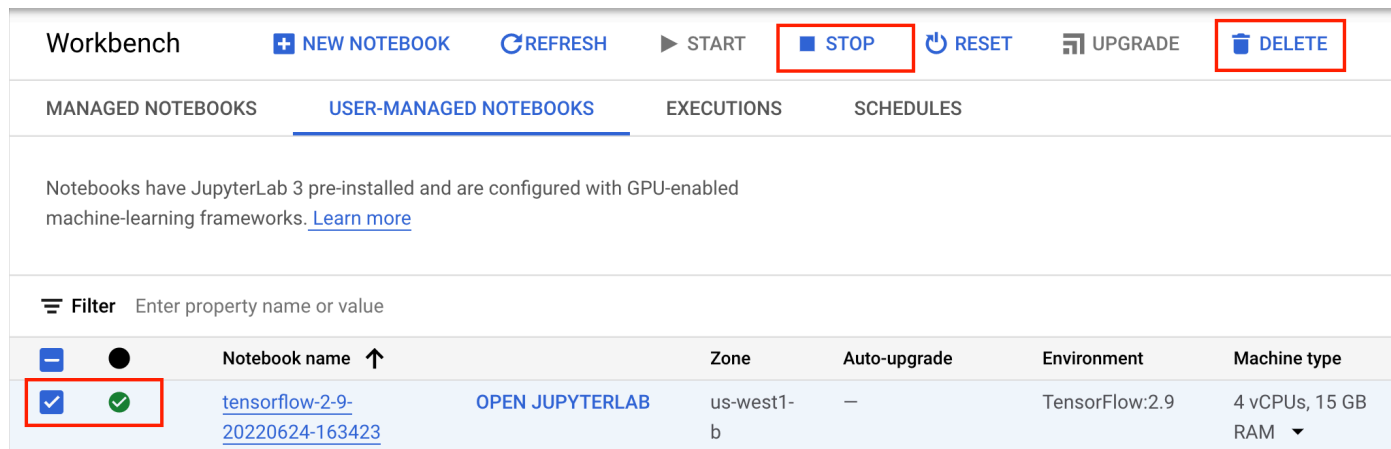
如要進一步瞭解 Vertex AI 的其他部分，請參閱[說明文件](#)。

步驟 8 · 約 1 分鐘

8. 清除所用資源

如要繼續使用在本實驗室中建立的筆記本，建議您在未使用時關閉筆記本。在 Google Cloud 控制台的 Workbench UI 中，選取筆記本，然後選取「停止」。

如要徹底刪除筆記本，請按一下右上方的「刪除」按鈕。



Workbench

[+ NEW NOTEBOOK](#) [REFRESH](#) [START](#) [STOP](#) [RESET](#) [UPGRADE](#) [DELETE](#)

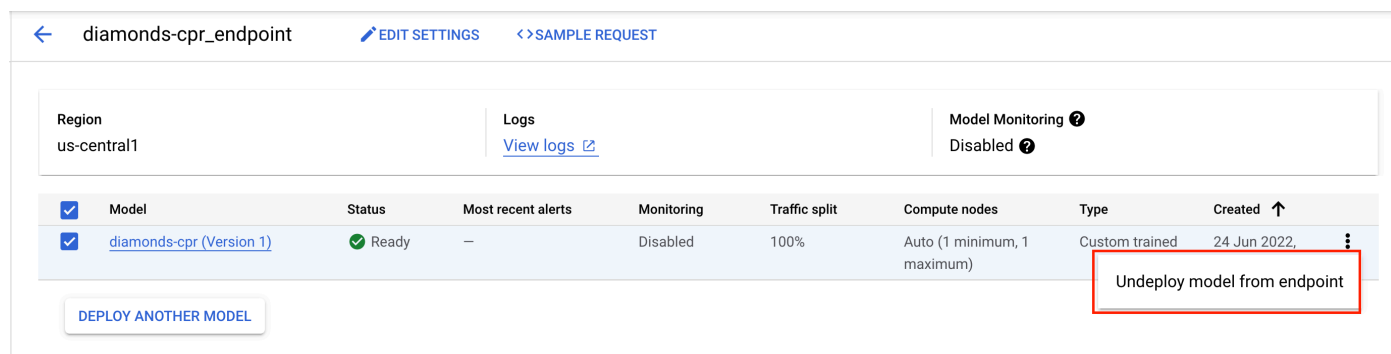
MANAGED NOTEBOOKS **USER-MANAGED NOTEBOOKS** EXECUTIONS SCHEDULES

Notebooks have JupyterLab 3 pre-installed and are configured with GPU-enabled machine-learning frameworks. [Learn more](#)

Filter Enter property name or value

	Notebook name ↑	Zone	Auto-upgrade	Environment	Machine type
☒	tensorflow-2-9-20220624-163423 OPEN JUPYTERLAB	us-west1-b	—	TensorFlow:2.9	4 vCPUs, 15 GB RAM

如要刪除已部署的端點，請前往控制台的「端點」部分，按一下您建立的端點，然後選取「Undeploy model from endpoint」（取消部署端點的模型）：



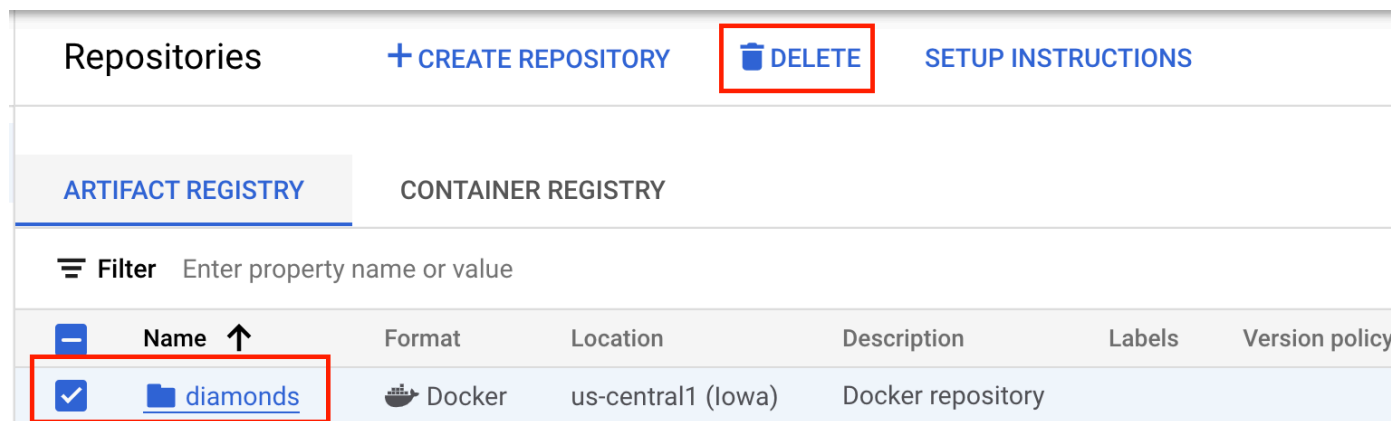
[← diamonds-cpr_endpoint](#) [EDIT SETTINGS](#) [SAMPLE REQUEST](#)

Region: us-central1 | Logs: [View logs](#) | Model Monitoring: Disabled

Model	Status	Most recent alerts	Monitoring	Traffic split	Compute nodes	Type	Created ↑
☒ diamonds-cpr (Version 1)	Ready	—	Disabled	100%	Auto (1 minimum, 1 maximum)	Custom trained	24 Jun 2022, 10:00 AM

[DEPLOY ANOTHER MODEL](#) [Undeploy model from endpoint](#)

如要刪除容器映像檔，請前往 Artifact Registry，選取您建立的存放區，然後選取「刪除」。



Repositories [+ CREATE REPOSITORY](#) [DELETE](#) [SETUP INSTRUCTIONS](#)

ARTIFACT REGISTRY CONTAINER REGISTRY

Filter Enter property name or value

Name ↑	Format	Location	Description	Labels	Version policy
☒ diamonds	Docker	us-central1 (Iowa)	Docker repository		

如要刪除 Storage Bucket，請使用 Cloud 控制台的導覽選單瀏覽至 Storage，選取 Bucket，然後按一下「Delete」（刪除）：

Storage browser

 CREATE BUCKET

 DELETE

 REFRESH

 Filter buckets

 Name	Created ↓
 bucket_of_fun	Sep 16, 2020, 11:16:30 AM